# Nourish: Documentation

**Hosted app:** https://nourishus.netlify.app/ (client app at `/`, nutritionist dashboard at `/dashboard`) **Code:** https://github.com/Yash2004Codes/nourish-appointments

## What we were asked

Build a small appointments product with two sides that share the same data. The client side is a mobile-first app: a home screen with the next appointment, booking with no double booking, upcoming and past appointments, and reschedule or cancel. The nutritionist side is a dashboard: a list of appointments, three colour-coded statuses (Booked, Done, No-show) that she can change, adding an appointment by hand, a "today" filter, and live sync with the client side. It also had to include at least one thing that helps with no-shows, be loaded with the five sample appointments, and I was asked to add anything important that was missing.

## My approach

I built it with Next.js, TypeScript and Tailwind. Both sides use the same set of API routes and the same database, so anything booked on one side shows up on the other (the dashboard refreshes every few seconds). Data is stored in Amazon DynamoDB, and the app is hosted on Netlify.

I wanted the client side to feel like a calm consumer app, not an admin screen: soft green and cream colours, a friendly home screen, a bottom tab bar on phones and a top bar on desktop. The dashboard is more practical: summary cards, filters, and a table on desktop that turns into cards on a phone.

## Decisions and assumptions

- **No login.** The brief says people should open it without one. A client is recognised by their mobile number, which the browser remembers after their first booking. The downside is that anyone who knows a number could see that person's visits. A real product would verify the number with a one-time code.
- **No double booking is enforced in the database, not just the screen.** Each booked slot has a lock record, saved in the same transaction as the appointment, and only one request can create it. I tested this by sending 5 to 8 bookings for the same slot at the same moment, several times against the real database and once on the live site: exactly one succeeded and the others got a polite "that time was just taken" message.
- **Slots:** 30 minutes each, 9:00 to 17:00, every day (one sample appointment is on a Sunday), bookable up to 30 days ahead. All times are Indian time.
- **A fourth status, "Cancelled".** The brief lists three statuses, but a cancelled visit has to go somewhere. It is shown in grey and hidden from the dashboard's default list, so history is kept.
- **Cancel and reschedule** only work on upcoming, booked visits. Rescheduling asks the client to confirm again.
- **How I handled the sample data:** the five rows used different formats. I read dates day-first and converted them all to one format (`24/10/2026`, `2026-10-24` and `25-10-2026` all become proper dates). Times like `11:30am`, `4 pm` and `14:00` became 24-hour times. Phone numbers lost their spaces, which also showed that the two Riya Shah rows are the same client. Meera Iyer has no phone number, so I left it blank ("No phone on file") instead of making one up. Each row kept the status it was given.

## What I added

- **Phone-number lookup for returning clients.** Without logins, "my appointments" has nothing to attach to. The phone number is how a client finds their own visits.
- **No-show helpers, beyond the one required:** a "Yes, I'll be there" confirmation button for clients; Confirmed / Not confirmed labels and a count of unconfirmed visits on the dashboard; a warning next to clients who have missed before; the overall no-show rate; and a button that opens a ready-written WhatsApp reminder.
- **Smaller extras:** an "add to calendar" file after booking, a "Reset demo data" link on the dashboard so the sample data can always be restored, and search by name or phone.

## What the AI got wrong

- **Tapping a time slot booked immediately.** The slot buttons were inside the booking form without being marked as plain buttons, so every tap submitted the form. I caught it because my automated browser test created a booking I never asked for. Fix: mark those buttons as `type="button"`, then re-test.
- **The desktop booking page spilled off the screen.** The scrolling row of days stretched the page layout and pushed the details panel out of view. I caught it in a screenshot. Fix: let those columns shrink (`min-w-0`).
- **A crash when many people booked at once.** In a stress test, one of eight simultaneous requests came back as a server error. The database reported a "transaction conflict", which the code didn't expect. It now treats that as "slot taken". I found it by firing parallel requests at the database, not by reading the code.
- **Wrong assumptions about the tooling.** The project template included experimental caching settings I didn't need, and a strict lint rule that flagged normal loading code. I removed the settings and relaxed the rule after the first build.
- **A hosting plan that needed changing.** I first planned AWS Amplify, but connecting it to GitHub needs an authorisation step that couldn't safely be done from the command line. I switched to Netlify. That meant giving the app limited database credentials (access to one table only) under custom variable names, because Netlify reserves names starting with `AWS_`.

## How to run it

```
 npm install
npm run dev
```

Open http://localhost:3000. With no setup, it uses in-memory data already loaded with the five sample appointments. To use the real database, set `DYNAMODB_TABLE`, `NOURISH_AWS_REGION`, `NOURISH_AWS_ACCESS_KEY_ID` and `NOURISH_AWS_SECRET_ACCESS_KEY` first. The table has a text partition key `pk` and a text sort key `sk`. The client app is at `/` and the dashboard at `/dashboard`. The "Reset demo data" link at the bottom of the dashboard restores the sample rows.